Deterministic Citation Enforcement in RAG

Complete End-to-End Implementation Guide

Large Language Models University Elective · Inference-Time Attention Intervention
Mehmet Can Özen — Student ID: 20210808020 · Revised March 2026

Hardware Target: ASUS Dual RTX 5070 (12 GB VRAM) · Intel Core Ultra 7 265KF · 64 GB DDR5 6800 MHz · 1 TB NVMe

---

Overview

Enterprise RAG systems suffer from *citation hallucination* — a model generates a confident answer and appends a source tag (e.g., [Doc 1]) to a claim that is absent from, or directly contradicted by, the retrieved document. This is distinct from factual hallucination: the model may retrieve the right document but cite the wrong one, or confabulate a plausible-sounding excerpt from parametric memory alone. In high-stakes domains (fintech, legal, healthcare), such failures constitute audit failures and regulatory violations.[1][2]

This project implements an **inference-time attention-thresholding mechanism** that replaces model.generate() with a custom autoregressive decoding loop. At every token step, the decoder monitors the cross-attention mass directed at the retrieved document; if this mass falls below a calibrated threshold for several consecutive tokens, generation is aborted and replaced by an explicit abstention signal. This converts silent hallucinations into detectable, auditable refusals.

> **Zero fine-tuning.** The entire system operates at inference time. No QLoRA adapter is trained and no gradient steps are taken. The mechanism is model-agnostic and can be layered onto any pre-trained decoder checkpoint without additional compute.

Project Directory Structure

Set this up before writing a single line of model code. A clean layout prevents import chaos and keeps evaluation scripts separate from core library code.

```
rag_attribution/
├── environment.yml            # Conda env spec (pinned versions)
├── requirements.txt           # pip fallback
├── README.md
├── .env.example               # HF_TOKEN placeholder
│
├── src/
│   └── rag_attribution/
│       ├── __init__.py
│       ├── model_loader.py     # load_quantized_model()
│       ├── vector_store.py     # build_or_load_vectorstore()
│       ├── retriever.py        # build_hybrid_retriever()
│       ├── attention.py        # aggregate_new_token_attention(),
│       │                       # get_context_attention_score(),
│       │                       # find_document_token_span()
│       ├── decoder.py          # AttributedRAGDecoder
│       ├── verifier.py         # verify_citation()
│       ├── evaluator.py        # evaluate_system(), bootstrap_ci()
│       ├── corpus.py           # build_finance_corpus()
│       └── prompts.py          # PROMPT_TEMPLATE, build_prompt()
│
├── scripts/
│   ├── 00_smoke_test.py        # Quick sanity check (5 queries, no GPU OOM)
│   ├── 01_calibrate_threshold.py
│   ├── 02_run_asqa_eval.py     # Overnight ASQA full evaluation
```

```
│   ├── 03_run_finance_eval.py   # Overnight finance corpus evaluation
│   ├── 04_ablation_sweep.py
│   └── 05_plot_figures.py
│
├── data/
│   ├── asqa/                   # Auto-downloaded via HF datasets
│   └── finance_corpus/         # Generated by corpus.py
│       ├── documents.parquet
│       └── queries.parquet
│
├── outputs/
│   ├── results/                # JSON result files per eval run
│   ├── figures/                # Ablation curves, histograms
│   └── checkpoints/            # Any saved decoder state
│
└── tests/
    ├── test_attention.py
    ├── test_decoder.py
    ├── test_verifier.py
    └── conftest.py             # Shared pytest fixtures
```

Phase 0 — Environment & Dependency Setup

0.1 Conda Environment

Pin every dependency. Floating versions are the number-one cause of silent regressions in transformer experiments.

```
conda create -n rag_attribution python=3.11 -y
conda activate rag_attribution
```

`environment.yml`:

```
name: rag_attribution
channels:
```

```yaml
  - pytorch
  - nvidia
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - pip
  - pip:
    - torch==2.5.1
    - torchvision==0.20.1
    --extra-index-url https://download.pytorch.org/whl/cu124
    - transformers==4.46.0       # last version w/ reliable output_attentions
    - accelerate==1.1.0
    - bitsandbytes==0.44.1
    - peft==0.13.0               # imported but NOT used — kept for env parity
    - langchain==0.3.7
    - langchain-community==0.3.5
    - chromadb==0.5.18
    - sentence-transformers==3.2.1
    - rank-bm25==0.2.2           # BM25 retriever backend
    - fuzzywuzzy==0.18.0
    - python-Levenshtein==0.25.1
    - datasets==3.0.2
    - evaluate==0.4.3
    - ragas==0.2.3               # optional secondary evaluation
    - scikit-learn==1.5.2
    - scipy==1.14.1
    - pandas==2.2.3
    - pyarrow==17.0.0
    - matplotlib==3.9.2
    - seaborn==0.13.2
    - python-dotenv==1.0.1
    - pytest==8.3.3
    - pytest-mock==3.14.0
    - tqdm==4.67.0
    - huggingface-hub==0.26.2
```

```
conda env create -f environment.yml
conda activate rag_attribution
```

0.2 CUDA Verification

```
# scripts/00_smoke_test.py  (first block)
import torch

assert torch.cuda.is_available(), "CUDA not available — check driver"
print(f"Device: {torch.cuda.get_device_name(0)}")
print(f"VRAM total: {torch.cuda.get_device_properties(0).total_memory / 1e9:
print(f"PyTorch: {torch.__version__}")
```

Expected output on your rig:

```
Device: NVIDIA GeForce RTX 5070
VRAM total: 12.0 GB
PyTorch: 2.5.1+cu124
```

0.3 HuggingFace Token

Llama-3 requires a gated download. Create `.env`:

```
HF_TOKEN=hf_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

```
# src/rag_attribution/model_loader.py  (top of file)
import os
from dotenv import load_dotenv
load_dotenv()
HF_TOKEN = os.getenv("HF_TOKEN")
```

Phase 1 — Data Preparation

1.1 ASQA Benchmark

ASQA (Attributed Short Questions and Answers) provides open-domain factoid questions grounded in Wikipedia passages. Use the `dev` split for primary evaluation and a 200-example slice of `train` for threshold calibration only — these must never overlap.

```python
# src/rag_attribution/evaluator.py
from datasets import load_dataset

def load_asqa_splits():
    """
    Returns (calib_set, val_set) as lists of dicts.
    calib_set: 200 train examples for threshold calibration only.
    val_set:   full dev split (~1015 examples) for primary evaluation.
    Never use calib_set data during the final metric computation.
    """
    asqa = load_dataset(
        "google-research-datasets/ASQA",
        trust_remote_code=True
    )
    calib_set = list(asqa["train"])[:200]
    val_set   = list(asqa["dev"])
    return calib_set, val_set
```

Each ASQA example has the structure:

```
{
  "ambiguous_question": "...",
  "qa_pairs": [
    {
      "question": "...",
      "short_answer": "...",
      "context": "... Wikipedia passage ..."
    }
```

```
  ],
  "annotations": [...]
}
```

1.2 Synthetic Finance Corpus

Generate 100 documents and 500 queries designed specifically to stress-test the attention mechanism under enterprise edge cases: overlapping numerical claims across companies, contradictory quarterly figures, and unanswerable queries that test whether abstention fires correctly.

```python
# src/rag_attribution/corpus.py
import numpy as np
import pandas as pd

def build_finance_corpus(
    num_docs: int = 100,
    num_queries: int = 500,
    unanswerable_fraction: float = 0.1,  # 10% queries have no matching doc
    seed: int = 42
) -> tuple[list[dict], list[dict]]:
    """
    Returns (docs, queries).
    - docs:    list of {id, text, company, quarter, year, revenue}
    - queries: list of {query, answer, source_doc, answerable}

    unanswerable queries contain a company/quarter combination that does
    NOT appear in docs — ideal for testing the abstention mechanism in
    isolation from citation accuracy.
    """
    rng       = np.random.default_rng(seed)
    companies = ["Acme Corp", "Globex Ltd", "Initech", "Umbrella Inc", "Hooli"
                 "Vehement Capital", "Soylent Corp", "Prestige Worldwide"]
    quarters  = ["Q1", "Q2", "Q3", "Q4"]
    years     = [2022, 2023, 2024]

    docs = []
```

```
for i in range(num_docs):
    co  = rng.choice(companies)
    qtr = rng.choice(quarters)
    yr  = int(rng.choice(years))
    rev = round(float(rng.uniform(10.0, 200.0)), 1)
    margin = round(float(rng.uniform(5.0, 35.0)), 1)
    yoy    = round(float(rng.uniform(-10.0, 30.0)), 1)
    doc_text = (
        f"{co} reported {qtr} {yr} revenue of ${rev}M, "
        f"representing a {yoy}% year-over-year change. "
        f"Operating margin was {margin}%. "
        f"Net income reached ${round(rev * margin / 100, 1)}M."
    )
    docs.append({
        "id": f"fin_{i:03d}", "text": doc_text,
        "company": co, "quarter": qtr, "year": yr, "revenue": rev
    })

queries = []
n_unanswerable = int(num_queries * unanswerable_fraction)

# Answerable queries
for _ in range(num_queries - n_unanswerable):
    doc = rng.choice(docs)
    queries.append({
        "query":      f"What was {doc['company']}'s revenue in {doc['quarter']} 
        "answer":     f"${doc['revenue']}M",
        "source_doc": doc["text"],
        "answerable": True
    })

# Unanswerable queries (combination absent from corpus)
phantom_company = "Phantom Industries"
for _ in range(n_unanswerable):
    qtr = rng.choice(quarters)
    yr  = int(rng.choice(years))
    queries.append({
        "query":      f"What was {phantom_company}'s revenue in {qtr} {yr}?",
```

```
        "answer":     None,
        "source_doc": None,
        "answerable": False
    })

    # Serialize for reproducibility
    pd.DataFrame(docs).to_parquet("data/finance_corpus/documents.parquet",
    pd.DataFrame(queries).to_parquet("data/finance_corpus/queries.parquet",
    print(f"Corpus built: {len(docs)} docs, {len(queries)} queries "
          f"({n_unanswerable} unanswerable)")
    return docs, queries
```

1.3 Embedding the Documents into ChromaDB

The embedding model runs on CPU to keep the GPU free for the LLM. BAAI/bge-large-en-v1.5 scores strongly on the MTEB benchmark and is well-suited to factoid QA tasks.

```
# src/rag_attribution/vector_store.py
import os
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter

EMBED_MODEL   = "BAAI/bge-large-en-v1.5"
PERSIST_DIR   = "./data/chroma_store"
CHUNK_SIZE    = 512
CHUNK_OVERLAP = 50

def get_embeddings():
    return HuggingFaceEmbeddings(
        model_name=EMBED_MODEL,
        model_kwargs={"device": "cpu"},
        encode_kwargs={"normalize_embeddings": True}  # required for cosine s
    )

def build_or_load_vectorstore(raw_documents=None):
    """
```

```
    If the persistent store already exists, load it to avoid re-embedding.
    Otherwise, split raw_documents and embed from scratch.
    raw_documents: list of LangChain Document objects.
    """
    embeddings = get_embeddings()

    if os.path.exists(PERSIST_DIR) and os.listdir(PERSIST_DIR):
        print(f"Loading existing ChromaDB from {PERSIST_DIR}")
        return Chroma(
            persist_directory=PERSIST_DIR,
            embedding_function=embeddings
        )

    if raw_documents is None:
        raise ValueError("No existing store found and no documents provided.'

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
        separators=["\n\n", "\n", ". ", " ", ""]
    )
    chunks = text_splitter.split_documents(raw_documents)
    print(f"Indexing {len(chunks)} chunks into ChromaDB...")
    vectorstore = Chroma.from_documents(
        chunks,
        embeddings,
        persist_directory=PERSIST_DIR
    )
    print(f"ChromaDB built and persisted to {PERSIST_DIR}")
    return vectorstore
```

Phase 2 — Hybrid Retriever (Optional Enhancement)

Pure dense retrieval with ChromaDB misses exact-match queries ("What was Acme Corp's Q3 2023 revenue?"). BM25 keyword retrieval fills this gap. An ensemble of BM25 + dense retrieval is the recommended production pattern as of 2025.

```python
# src/rag_attribution/retriever.py
from langchain.retrievers import BM25Retriever, EnsembleRetriever

def build_hybrid_retriever(vectorstore, all_chunks, k: int = 5):
    """
    Combines dense vector retrieval and BM25 keyword retrieval.
    Weights: 50% dense + 50% BM25 (adjust if one domain dominates).

    Args:
        vectorstore: Chroma vectorstore with embedded chunks.
        all_chunks:  The same list of Document objects used to build the store.
        k:           Number of results each sub-retriever returns.

    Returns:
        EnsembleRetriever that merges and deduplicates results.
    """
    dense_retriever = vectorstore.as_retriever(search_kwargs={"k": k})
    bm25_retriever  = BM25Retriever.from_documents(all_chunks)
    bm25_retriever.k = k

    ensemble = EnsembleRetriever(
        retrievers=[dense_retriever, bm25_retriever],
        weights=[0.5, 0.5]
    )
    return ensemble
```

> **Why not just dense?** BM25 excels at exact numeric queries ("$47.3M") and entity matching, while dense retrieval handles paraphrase and semantic equivalence. For a finance corpus dominated by specific dollar figures, BM25 recall is non-trivial.

Phase 3 — Model Loading

3.1 4-bit NF4 Quantisation

4-bit NF4 quantisation via bitsandbytes reduces Llama-3-8B from ~28 GB to approximately 5.5 GB VRAM, leaving headroom for the KV cache, attention tensors, and ChromaDB embeddings on the same 12 GB device.

```
# src/rag_attribution/model_loader.py
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAnd

MODEL_ID = "meta-llama/Meta-Llama-3-8B-Instruct"

def load_quantized_model():
    """
    Load Llama-3-8B-Instruct in 4-bit NF4 quantisation.

    CRITICAL: attn_implementation="eager" is mandatory.
    Flash Attention 2 does not return attention weight tensors.
    Without eager mode, output_attentions=True either raises
    NotImplementedError or silently returns None.
    """
    bnb_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_use_double_quant=True   # saves ~0.4 GB extra VRAM
    )

    tokenizer = AutoTokenizer.from_pretrained(
        MODEL_ID,
```

```
    token=HF_TOKEN
)
tokenizer.pad_token = tokenizer.eos_token   # avoid warnings

model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    quantization_config=bnb_config,
    device_map="cuda:0",
    attn_implementation="eager",   # ← MANDATORY for output_attentions
    token=HF_TOKEN
)
model.eval()

# Sanity check
allocated = torch.cuda.memory_allocated() / 1e9
print(f"Model loaded. VRAM used: {allocated:.2f} GB")
assert allocated < 8.0, f"VRAM too high ({allocated:.2f} GB) — check config

return model, tokenizer
```

3.2 VRAM Budget

Component	Approx. VRAM
Llama-3-8B NF4 (double quant)	~5.1 GB
KV cache (3500-token context)	~1.4 GB
Attention tensors (1 × N, last 4 layers)	~6 MB
ChromaDB embeddings (CPU)	0 GB GPU
Buffer / OS overhead	~1.0 GB
Total	**~7.5 GB**

This leaves approximately 4.5 GB of headroom on the 12 GB card — enough to handle occasional spikes without triggering the CodecSep GPU conflict.

Phase 4 — Prompt Engineering

The prompt template must be held **identical** across the constrained system and the Vanilla RAG baseline. Any variation confounds the comparison.

```
# src/rag_attribution/prompts.py

PROMPT_TEMPLATE = """\
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
You are a precise research assistant. Answer the question using ONLY
the information provided in the context below. If the context does not
contain sufficient information to answer, explicitly say so.
<|eot_id|>
<|start_header_id|>user<|end_header_id|>
Context:
{context}

Question: {question}
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
"""


def build_prompt(question: str, retrieved_docs: list) -> str:
    """
    Join retrieved document page_content with double newlines.
    This preserves chunk boundaries and makes them detectable
    by find_document_token_span().
    """
    context = "\n\n".join([d.page_content for d in retrieved_docs])
    return PROMPT_TEMPLATE.format(context=context, question=question)
```

> **Llama-3 special tokens matter.** Use the full `<|begin_of_text|>...<|eot_id|>` chat format. The vanilla `"Context:\n{context}\n\nQuestion: {question}\nAnswer:"` template works but produces lower-quality outputs. The chat template also produces more predictable attention patterns in the final layers.

Phase 5 — Memory-Efficient Attention Extraction

This is the technical core of the project. All three functions must be correct before the decoder can be built.

5.1 The OOM Trap

The naive approach — calling `model(input_ids, output_attentions=True)` at every decoding step without KV caching — computes the full $N \times N$ attention matrix across all 32 transformer layers. For a 3,000-token RAG context, this is approximately 18.4 GB, which will immediately crash a 12 GB GPU.

The fix: After the initial prefill pass, feed only the *single newly generated token* alongside the accumulated `past_key_values`. Each attention tensor shrinks from shape `(batch, 32, N, N)` ≈ 18.4 GB to shape `(batch, 32, 1, N)` ≈ 6 MB — a five-order-of-magnitude reduction.

```python
# src/rag_attribution/attention.py
import torch
import torch.nn.functional as F


def aggregate_new_token_attention(
    attention_tuple: tuple,
    num_tail_layers: int = 4
) -> torch.Tensor:
    """
    Average attention from the last `num_tail_layers` transformer layers
    for the newly generated token only.
```

```
    Why last 4 layers?
    Early layers encode positional/syntactic structure.
    Final layers (28-31 in Llama-3-8B) encode high-level semantic grounding.
    Averaging over the last 4 layers + all heads yields a stable signal
    that tracks whether the model is reading the document or confabulating.

    Args:
        attention_tuple: Each element has shape (batch, heads, 1, seq_len)
                         when using KV cache + single new token.
        num_tail_layers: Default 4 (layers 28-31 for Llama-3-8B).

    Returns:
        1-D tensor (seq_len,) — normalised attention over the full context.
    """
    tail = attention_tuple[-num_tail_layers:]                # last N layers
    stacked = torch.stack(tail)                              # (N, batch, heads, 1, seq_len)
    avg = stacked.mean(dim=[0, 2])                           # avg over layers & heads
    avg = avg.squeeze()                                      # → (seq_len,)
    return F.softmax(avg.float(), dim=-1)                    # re-normalise after ave


def get_context_attention_score(
    attention_weights: torch.Tensor,
    context_start: int,
    context_end: int
) -> float:
    """
    Sum normalised attention over the retrieved document token span.

    Returns a scalar in [0, 1]:
      1.0 → token attends exclusively to retrieved document
      0.0 → token attends entirely to the instruction prefix

    Args:
        attention_weights: 1-D tensor (seq_len,) from aggregate_new_token_atte
        context_start:     First token index of the document in the prompt.
        context_end:       One past the last token index of the document.
```

```
    """
    if context_end <= context_start:
        raise ValueError(
            f"Invalid context span: [{context_start}, {context_end}]. "
            f"Did find_document_token_span() fail?"
        )
    return attention_weights[context_start:context_end].sum().item()


def find_document_token_span(
    full_prompt: str,
    document_text: str,
    tokenizer
) -> tuple[int, int]:
    """
    Return (start_idx, end_idx) — exclusive — of `document_text` within
    the tokenised `full_prompt`.

    Strategy: character-level substring find, then tokenise prefix
    and document separately to compute token offsets.
    This avoids edge cases where special tokens or subword splits
    shift indices unexpectedly.

    Raises ValueError if document_text is not found verbatim in full_prompt.
    """
    char_start = full_prompt.find(document_text)
    if char_start == -1:
        raise ValueError(
            "Document text not found verbatim in prompt. "
            "Ensure build_prompt() joins documents before passing to decoder."
        )
    prefix     = full_prompt[:char_start]
    prefix_ids = tokenizer(prefix, add_special_tokens=True).input_ids
    doc_ids    = tokenizer(document_text, add_special_tokens=False).input_ids

    start = len(prefix_ids)
```

```
    end   = start + len(doc_ids)
    return start, end
```

5.2 Layer-Range Sensitivity

If grounding scores are consistently near zero (model never clears the threshold), the first debugging step is to widen the layer range to the last 8 layers (`num_tail_layers=8`). Over-narrow layer selection can miss the semantic signal in models that encode grounding earlier than expected.

Phase 6 — Custom Decoding Loop (Core Contribution)

The `AttributedRAGDecoder` is the primary novel contribution of the project. It replaces HuggingFace's `model.generate()` with a hand-rolled autoregressive loop that checks the grounding score at every token step.

```
# src/rag_attribution/decoder.py
import torch
from .attention import (
    aggregate_new_token_attention,
    get_context_attention_score,
    find_document_token_span
)


class AttributedRAGDecoder:
    """
    Autoregressive decoder with hard cross-attention grounding enforcement

    Algorithm:
      1. PREFILL: Run the full prompt through the model once, extracting
         the initial KV cache and first predicted token.
      2. LOOP: Feed only the new token + KV cache at each step.
         Shape: (batch, heads, 1, N) — ~6 MB vs ~18.4 GB naive.
```

```
  3. THRESHOLD CHECK: If grounding score < threshold for
     max_consecutive_failures consecutive tokens, abort and return
     an explicit abstention message. The failure counter resets on
     any token that clears the threshold — this prevents false
     positives on conjunctions and punctuation.
  4. COMMIT: Otherwise, greedily accept the token and continue.

Returns a structured result dict — not a raw string — for clean
integration with the evaluation loop and logging.
"""

ABSTAIN_MSG = (
    " [SYSTEM: Generation aborted — "
    "insufficient grounding in retrieved source.]"
)

def __init__(
    self,
    model,
    tokenizer,
    attention_threshold: float = 0.3,
    max_consecutive_failures: int = 3,
    num_tail_layers: int = 4
):
    self.model        = model
    self.tokenizer    = tokenizer
    self.threshold    = attention_threshold
    self.max_failures = max_consecutive_failures
    self.n_layers     = num_tail_layers

def decode_with_attribution(
    self,
    context_prompt: str,
    doc_start_idx: int,
    doc_end_idx: int,
    max_new_tokens: int = 200
) -> dict:
    """
```

```
    Run attributed generation.

    Returns dict:
      text           — decoded output string (may end with ABSTAIN_MSG)
      aborted        — True if generation was terminated early
      grounding_trace — list[float] of per-token grounding scores
      failure_steps   — list[int] of token indices where threshold was violate
      prompt_tokens   — int, number of input tokens (useful for logging)
    """
    enc = self.tokenizer(
        context_prompt,
        return_tensors="pt",
        truncation=True,
        max_length=3500   # leave headroom; hard cap prevents OOM
    ).to(self.model.device)

    input_ids       = enc.input_ids
    generated_ids   = []
    grounding_trace = []
    failure_steps   = []
    failure_count   = 0

    # ── PREFILL ──────────────────────────────────────────────────────
    with torch.no_grad():
        out = self.model(
            input_ids,
            use_cache=True,
            output_attentions=True
        )
    past_kv = out.past_key_values

    # Extract grounding for the last prefill position
    prefill_attn  = aggregate_new_token_attention(out.attentions, self.n_laye
    grounding_trace.append(
        get_context_attention_score(prefill_attn, doc_start_idx, doc_end_idx)
    )

    next_token = torch.argmax(out.logits[:, -1, :], dim=-1, keepdim=True)
```

```
        generated_ids.append(next_token.item())

        # ── AUTOREGRESSIVE LOOP ──────────────────────────────────────
        for step in range(max_new_tokens - 1):
            with torch.no_grad():
                out = self.model(
                    next_token,            # only the single new token
                    use_cache=True,
                    past_key_values=past_kv,
                    output_attentions=True
                )
            past_kv = out.past_key_values   # grow cache by 1 token

            attn  = aggregate_new_token_attention(out.attentions, self.n_layers)
            score = get_context_attention_score(attn, doc_start_idx, doc_end_idx)
            grounding_trace.append(score)

            # ── THRESHOLD CHECK ──────────────────────────────────────
            if score < self.threshold:
                failure_count += 1
                failure_steps.append(step + 1)
                if failure_count >= self.max_failures:
                    decoded = self.tokenizer.decode(
                        generated_ids, skip_special_tokens=True
                    )
                    return {
                        "text":           decoded + self.ABSTAIN_MSG,
                        "aborted":        True,
                        "grounding_trace": grounding_trace,
                        "failure_steps":   failure_steps,
                        "prompt_tokens":   input_ids.shape[-1]
                    }
            else:
                failure_count = 0   # reset on any successful grounding

            # ── COMMIT TOKEN ─────────────────────────────────────────
            next_token = torch.argmax(out.logits[:, -1, :], dim=-1, keepdim=True)
            generated_ids.append(next_token.item())
```

```
        if next_token.item() == self.tokenizer.eos_token_id:
            break

    return {
        "text":           self.tokenizer.decode(generated_ids, skip_special_tokens=
        "aborted":        False,
        "grounding_trace": grounding_trace,
        "failure_steps":   failure_steps,
        "prompt_tokens":   input_ids.shape[-1]
    }
```

6.1 Why Three Consecutive Failures?

A single low-attention token does not signal hallucination. Conjunctions ("and", "but"), punctuation, and discourse markers naturally receive low cross-document attention because they are function words predicted almost entirely from syntactic context. The counter resets on any token that clears the threshold, so `max_consecutive_failures=3` catches sustained confabulation while absorbing brief function-word dips.

6.2 The Vanilla RAG Baseline

Reproduce under identical conditions — same seed, temperature, `top_p`, retrieved documents. Do **not** use `model.generate()` for the baseline if you want grounding traces; instead, run the decoder without the threshold check by setting `attention_threshold=0.0`.

```
# Baseline: threshold disabled
baseline_decoder = AttributedRAGDecoder(
    model, tokenizer,
    attention_threshold=0.0,        # never aborts
    max_consecutive_failures=99999
)
```

Phase 7 — Post-Hoc Citation Verification

Fuzzy string matching with `fuzzywuzzy.partial_ratio` is robust to minor paraphrasing — the model may rephrase a claim slightly while preserving factual content. A threshold of 80 is standard for citation verification tasks.

```
# src/rag_attribution/verifier.py
from fuzzywuzzy import fuzz


def verify_citation(
    generated_claim: str,
    source_document: str,
    fuzzy_threshold: int = 80
) -> dict:
    """
    Post-hoc verification: does the generated claim appear in the source?

    Uses partial_ratio rather than ratio or token_sort_ratio because
    generated answers are typically shorter than the full source document —
    partial_ratio checks if the claim is a substring-level match.

    Returns:
        {"score": int (0-100), "verified": bool}
    """
    score = fuzz.partial_ratio(
        generated_claim.lower(),
        source_document.lower()
    )
    return {"score": score, "verified": score >= fuzzy_threshold}


def extract_claims(generated_text: str) -> list[str]:
    """
    Split generated text into sentence-level claims for fine-grained verificatior
    Sentences shorter than 10 characters are filtered (likely artifacts).
```

```
    """
    sentences = [s.strip() for s in generated_text.split(".") if len(s.strip()) >= 10]
    return sentences
```

Phase 8 — Evaluation Framework

8.1 Primary Evaluation Loop

Abstentions are penalised as failures in Citation Accuracy. They cannot be excluded from the denominator — doing so would inflate the metric dishonestly versus the baseline.

```
# src/rag_attribution/evaluator.py
import json
from pathlib import Path
from tqdm import tqdm
from .verifier import verify_citation
from .prompts import build_prompt
from .attention import find_document_token_span


def evaluate_system(
    decoder,
    dataset: list[dict],
    retriever,
    tokenizer,
    output_path: str = None,
    max_new_tokens: int = 200
) -> dict:
    """
    Run the full evaluation loop on a dataset.

    Args:
        decoder:     AttributedRAGDecoder instance (constrained or baseline).
        dataset:     List of dicts with at least "query" and "source_doc" keys.
        retriever:   LangChain retriever (hybrid or dense-only).
        tokenizer:   HuggingFace tokenizer.
```

```
    output_path:  If provided, save per-query results as JSON.

Returns dict:
    citation_accuracy    — fraction of queries with verified citation
    abstention_rate      — fraction of queries that triggered abstention
    mean_grounding_score — average per-query grounding score
    per_query_results    — full list for inspection and error analysis
"""
per_query = []

for item in tqdm(dataset, desc="Evaluating"):
    docs     = retriever.get_relevant_documents(item["query"])
    doc_text = "\n\n".join([d.page_content for d in docs])
    prompt   = build_prompt(item["query"], docs)

    try:
        doc_start, doc_end = find_document_token_span(prompt, doc_text, tok
    except ValueError as e:
        per_query.append({"error": str(e), "query": item["query"]})
        continue

    result = decoder.decode_with_attribution(
        prompt, doc_start, doc_end, max_new_tokens=max_new_tokens
    )

    mean_score = (
        sum(result["grounding_trace"]) / max(len(result["grounding_trace"]), 1
    )

    if not result["aborted"]:
        verification = verify_citation(result["text"], doc_text)
        citation_ok  = verification["verified"]
    else:
        citation_ok = False   # abstentions count as failures

    per_query.append({
        "query":         item["query"],
        "generated":     result["text"],
```

```
        "aborted":       result["aborted"],
        "citation_ok":    citation_ok,
        "mean_grounding": mean_score,
        "failure_steps":  result["failure_steps"],
        "prompt_tokens":  result.get("prompt_tokens", -1)
    })

n = len(per_query)
metrics = {
    "citation_accuracy":    sum(r["citation_ok"]  for r in per_query if "citation
    "abstention_rate":      sum(r["aborted"]      for r in per_query if "aborted"
    "mean_grounding_score": sum(r["mean_grounding"] for r in per_query
    "n_queries":            n,
    "per_query_results":    per_query
}

if output_path:
    Path(output_path).parent.mkdir(parents=True, exist_ok=True)
    with open(output_path, "w") as f:
        json.dump(metrics, f, indent=2, default=str)

return metrics
```

8.2 Threshold Ablation Study

Sweep `attention_threshold` from 0.10 to 0.50 in steps of 0.05. Record Citation Accuracy and Abstention Rate at each value. This produces the precision-recall tradeoff curve required for the ablation section of the final report.

```
# scripts/04_ablation_sweep.py
import numpy as np
import pandas as pd
from src.rag_attribution.model_loader import load_quantized_model
from src.rag_attribution.decoder import AttributedRAGDecoder
from src.rag_attribution.evaluator import evaluate_system, load_asqa_splits

model, tokenizer = load_quantized_model()
```

```python
calib_set, _     = load_asqa_splits()
# Use calibration set only — never the val set — for threshold selection
retriever = ...   # load your hybrid retriever here

thresholds = np.round(np.arange(0.10, 0.55, 0.05), 2)
records    = []

for t in thresholds:
    decoder_t = AttributedRAGDecoder(
        model, tokenizer,
        attention_threshold=float(t),
        max_consecutive_failures=3
    )
    metrics = evaluate_system(
        decoder_t, calib_set[:50], retriever, tokenizer  # small slice for speed
    )
    records.append({
        "threshold":         t,
        "citation_accuracy": metrics["citation_accuracy"],
        "abstention_rate":   metrics["abstention_rate"],
        "mean_grounding":    metrics["mean_grounding_score"]
    })
    print(
        f"t={t:.2f}  "
        f"acc={metrics['citation_accuracy']:.3f}  "
        f"abstain={metrics['abstention_rate']:.3f}  "
        f"grounding={metrics['mean_grounding_score']:.3f}"
    )

df = pd.DataFrame(records)
df.to_csv("outputs/results/ablation_sweep.csv", index=False)
print(df.to_string())
```

8.3 Statistical Significance Testing

Use a **paired Wilcoxon signed-rank test** (preferred over paired t-test because citation accuracy scores are bounded and non-normal) plus 95% bootstrapped confidence intervals by resampling the test set 1,000 times.

```python
# src/rag_attribution/evaluator.py  (additional functions)
from scipy import stats
import numpy as np


def bootstrap_ci(
    scores: list[float],
    n_boot: int = 1000,
    ci: float = 0.95
) -> tuple[float, float, float]:
    """
    Returns (mean, lower_bound, upper_bound) for the given scores.
    """
    boots = [
        np.mean(np.random.choice(scores, len(scores), replace=True))
        for _ in range(n_boot)
    ]
    alpha = (1 - ci) / 2
    lo = float(np.percentile(boots, alpha * 100))
    hi = float(np.percentile(boots, (1 - alpha) * 100))
    return float(np.mean(scores)), lo, hi


def run_significance_test(
    constrained_scores: list[float],
    baseline_scores: list[float]
) -> dict:
    """
    One-sided Wilcoxon: H1 = constrained > baseline.
    Returns p-value and 95% CI for the constrained system.
    """
```

```
    stat, p = stats.wilcoxon(
        constrained_scores, baseline_scores,
        alternative="greater"
    )
    mean, lo, hi = bootstrap_ci(constrained_scores)
    return {
        "wilcoxon_statistic": stat,
        "p_value":            p,
        "significant":        p < 0.05,
        "constrained_mean":   mean,
        "ci_lower":           lo,
        "ci_upper":           hi
    }
```

8.4 RAGAS Secondary Evaluation (Optional)

RAGAS provides reference-free RAG evaluation metrics — Faithfulness, Context Recall, Answer Relevancy — without requiring ground truth for every query. Use it as a sanity check alongside your primary Citation Accuracy metric.

```
from ragas import evaluate as ragas_evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_recall
from datasets import Dataset

def run_ragas_eval(queries, answers, contexts, ground_truths):
    data = Dataset.from_dict({
        "question":   queries,
        "answer":     answers,
        "contexts":   [[c] for c in contexts],
        "ground_truth": ground_truths
    })
    result = ragas_evaluate(
        data,
        metrics=[faithfulness, answer_relevancy, context_recall]
    )
    return result
```

Phase 9 — Unit Tests

Unit tests are mandatory. They let you debug the attention and span-finding logic on CPU-only synthetic data before ever loading the 8B model.

```python
# tests/conftest.py
import pytest
import torch

@pytest.fixture
def synthetic_attention_tuple():
    """
    32 layers of (batch=1, heads=32, seq=1, seq_len=100) attention tensors.
    Simulates a single-token forward pass output with KV cache.
    Uniform attention: each token gets 1/100 weight.
    """
    layer = torch.ones(1, 32, 1, 100) / 100.0
    return tuple([layer] * 32)

@pytest.fixture
def peaked_attention_tuple():
    """
    Attention peaked over tokens 20-40 (the 'document' span).
    Last 4 layers have 0.7 mass in [20,40]; earlier layers are uniform.
    """
    uniform = torch.ones(1, 32, 1, 100) / 100.0
    peaked  = torch.zeros(1, 32, 1, 100)
    peaked[:, :, :, 20:40] = 0.70 / 20   # 0.70 total in doc span
    peaked[:, :, :, :20]   = 0.30 / 20   # 0.30 in prefix
    peaked[:, :, :, 40:]   = 0.00
    return tuple([uniform] * 28 + [peaked] * 4)
```

```python
# tests/test_attention.py
import torch
import pytest
```

```python
from src.rag_attribution.attention import (
    aggregate_new_token_attention,
    get_context_attention_score,
    find_document_token_span
)



def test_aggregate_sums_to_one(synthetic_attention_tuple):
    weights = aggregate_new_token_attention(synthetic_attention_tuple, num_
    assert abs(weights.sum().item() - 1.0) < 1e-5

def test_aggregate_shape(synthetic_attention_tuple):
    weights = aggregate_new_token_attention(synthetic_attention_tuple, num_
    assert weights.shape == (100,), f"Expected (100,), got {weights.shape}"

def test_peaked_grounding_score(peaked_attention_tuple):
    weights = aggregate_new_token_attention(peaked_attention_tuple, num_ta
    score   = get_context_attention_score(weights, 20, 40)
    assert score > 0.5, f"Expected score > 0.5 for peaked attention, got {score:.3

def test_uniform_grounding_score(synthetic_attention_tuple):
    weights = aggregate_new_token_attention(synthetic_attention_tuple, num_
    score   = get_context_attention_score(weights, 0, 100)
    assert abs(score - 1.0) < 1e-4, f"Expected ~1.0 for full-span uniform, got {sc

def test_invalid_span_raises():
    weights = torch.ones(100) / 100
    with pytest.raises(ValueError):
        get_context_attention_score(weights, 50, 30)   # end < start

def test_find_document_token_span(mocker):
    """Test with a real (small) tokenizer — mock is not needed here."""
    from transformers import AutoTokenizer
    tokenizer = AutoTokenizer.from_pretrained("gpt2")

    full_prompt   = "Question: Who wrote Hamlet? Context: Shakespeare wrot
    document_text = "Shakespeare wrote Hamlet."
    start, end = find_document_token_span(full_prompt, document_text, toker
```

```
    assert start < end
    # Verify the span reconstructs the document
    all_ids  = tokenizer(full_prompt).input_ids
    doc_ids  = tokenizer(document_text, add_special_tokens=False).input_ids
    assert all_ids[start:end] == doc_ids
```

```
# tests/test_decoder.py
import pytest
import torch
from unittest.mock import MagicMock, patch
from src.rag_attribution.decoder import AttributedRAGDecoder


@pytest.fixture
def mock_model_and_tokenizer():
    """
    Minimal mocks that allow the decoder to run 5 steps without GPU.
    The model always returns a grounding score above 0.3 (token index 42).
    """
    tokenizer = MagicMock()
    tokenizer.return_value = MagicMock(
        input_ids=torch.zeros(1, 10, dtype=torch.long)
    )
    tokenizer.eos_token_id = 2
    tokenizer.decode.return_value = "This is a generated answer."

    # Build a fake attention tuple: 32 layers, (1, 32, 1, 10)
    layer = torch.ones(1, 32, 1, 10) / 10.0
    fake_attentions = tuple([layer] * 32)

    # Logits: always predict token 42 (not EOS)
    fake_logits = torch.zeros(1, 1, 50257)
    fake_logits[0, 0, 42] = 100.0

    out = MagicMock()
    out.past_key_values = MagicMock()
    out.attentions      = fake_attentions
```

```
    out.logits          = fake_logits

    model = MagicMock()
    model.return_value = out
    model.device       = "cpu"

    return model, tokenizer


def test_decoder_does_not_abort_high_grounding(mock_model_and_toke
    model, tokenizer = mock_model_and_tokenizer
    decoder = AttributedRAGDecoder(
        model, tokenizer,
        attention_threshold=0.05,   # very low — should never abort
        max_consecutive_failures=3
    )
    result = decoder.decode_with_attribution(
        context_prompt="mock prompt",
        doc_start_idx=0,
        doc_end_idx=10,
        max_new_tokens=5
    )
    assert result["aborted"] is False
    assert len(result["grounding_trace"]) > 0


def test_decoder_aborts_on_low_grounding(mock_model_and_tokenizer):
    model, tokenizer = mock_model_and_tokenizer
    decoder = AttributedRAGDecoder(
        model, tokenizer,
        attention_threshold=0.99,   # impossibly high — always aborts
        max_consecutive_failures=3
    )
    result = decoder.decode_with_attribution(
        context_prompt="mock prompt",
        doc_start_idx=0,
        doc_end_idx=10,
        max_new_tokens=20
```

```
    )
    assert result["aborted"] is True
    assert decoder.ABSTAIN_MSG in result["text"]
    assert len(result["failure_steps"]) >= 3
```

Run all tests with:

```
pytest tests/ -v --tb=short
```

Phase 10 — Running the Full Evaluation

10.1 Overnight Script Structure

Full ASQA evaluation (1,015 queries × ~200 tokens each at ~3 seconds/query) takes approximately 50 minutes. Schedule as an overnight run to avoid GPU conflicts with CodecSep.

```
# scripts/02_run_asqa_eval.py
import json
from pathlib import Path
from src.rag_attribution.model_loader import load_quantized_model
from src.rag_attribution.vector_store import build_or_load_vectorstore
from src.rag_attribution.retriever import build_hybrid_retriever
from src.rag_attribution.decoder import AttributedRAGDecoder
from src.rag_attribution.evaluator import (
    evaluate_system, load_asqa_splits, run_significance_test
)

OPTIMAL_THRESHOLD = 0.30   # replace with calibrated value from ablation

model, tokenizer = load_quantized_model()
vectorstore      = build_or_load_vectorstore()
retriever        = build_hybrid_retriever(vectorstore, all_chunks=[])

_, val_set = load_asqa_splits()
```

```
# Constrained system
constrained_decoder = AttributedRAGDecoder(
    model, tokenizer,
    attention_threshold=OPTIMAL_THRESHOLD,
    max_consecutive_failures=3
)
constrained_metrics = evaluate_system(
    constrained_decoder, val_set, retriever, tokenizer,
    output_path="outputs/results/asqa_constrained.json"
)

# Vanilla RAG baseline
baseline_decoder = AttributedRAGDecoder(
    model, tokenizer,
    attention_threshold=0.0,
    max_consecutive_failures=99999
)
baseline_metrics = evaluate_system(
    baseline_decoder, val_set, retriever, tokenizer,
    output_path="outputs/results/asqa_baseline.json"
)

# Statistical test
constrained_scores = [
    r["citation_ok"] * 1.0
    for r in constrained_metrics["per_query_results"]
    if "citation_ok" in r
]
baseline_scores = [
    r["citation_ok"] * 1.0
    for r in baseline_metrics["per_query_results"]
    if "citation_ok" in r
]

sig_result = run_significance_test(constrained_scores, baseline_scores)
print(json.dumps(sig_result, indent=2))

print(f"\nConstrained — Citation Accuracy: {constrained_metrics['citation_ac
```

```
print(f"Baseline    — Citation Accuracy: {baseline_metrics['citation_accuracy']
print(f"Wilcoxon p = {sig_result['p_value']:.4f}")
```

10.2 Figures

```
# scripts/05_plot_figures.py
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json

# Figure 1: Ablation Curve
ablation = pd.read_csv("outputs/results/ablation_sweep.csv")
fig, ax1 = plt.subplots(figsize=(8, 4))
ax2 = ax1.twinx()
ax1.plot(ablation["threshold"], ablation["citation_accuracy"],
         "b-o", label="Citation Accuracy")
ax2.plot(ablation["threshold"], ablation["abstention_rate"],
         "r--s", label="Abstention Rate")
ax1.set_xlabel("Attention Threshold")
ax1.set_ylabel("Citation Accuracy", color="b")
ax2.set_ylabel("Abstention Rate", color="r")
ax1.set_title("Figure 1: Precision-Recall Tradeoff (Attention Threshold Sweep
fig.legend(loc="center right", bbox_to_anchor=(0.85, 0.5))
plt.tight_layout()
plt.savefig("outputs/figures/fig1_ablation_curve.pdf", dpi=300)

# Figure 2: Grounding Score Distribution
with open("outputs/results/asqa_constrained.json") as f:
    constrained = json.load(f)
with open("outputs/results/asqa_baseline.json") as f:
    baseline = json.load(f)

c_scores = [r["mean_grounding"] for r in constrained["per_query_results"] if
b_scores = [r["mean_grounding"] for r in baseline["per_query_results"]    if "r

fig, ax = plt.subplots(figsize=(8, 4))
```

```
ax.hist(b_scores, bins=40, alpha=0.6, label="Vanilla RAG", color="red")
ax.hist(c_scores, bins=40, alpha=0.6, label="Constrained",  color="blue")
ax.set_xlabel("Mean Per-Query Grounding Score")
ax.set_ylabel("Count")
ax.set_title("Figure 2: Grounding Score Distribution (ASQA dev)")
ax.legend()
plt.tight_layout()
plt.savefig("outputs/figures/fig2_grounding_distribution.pdf", dpi=300)
```

Phase 11 — Target Metrics and Success Criteria

Metric	Target	Definition
Citation Accuracy	> 95%	% of queries where the generated claim is present in the cited document (fuzzy match ≥ 80)
False Attribution Rate	< 5%	% of citations where the model attributes a claim to a non-supporting document
Abstention Rate	< 10%	% of queries aborted by the threshold check (counted as failures)
Mean Grounding Score	> 0.40	Average per-query attention mass directed at retrieved document tokens
Answer Quality Drop	< 10 pp vs baseline	Difference in correct-answer rate between constrained and Vanilla RAG

> **Note on citation accuracy baselines:** Research suggests that citation accuracy in RAG systems without explicit attribution training averages only 65–70%. A target of > 95% represents a substantial improvement and is achievable with this mechanism if the threshold is well-calibrated.

Phase 12 — Risk Register and Mitigations

Risk	Likelihood	Impact	Mitigation
GPU OOM during attention extraction	High without KV cache	Critical	Enforce past_key_values; cap input at 3,500 tokens; reduce num_tail_layers=2 as last resort
attn_implementation conflict	Medium	High	Pin transformers==4.46.0; always set attn_implementation="eager"
Grounding scores uniformly low	Low–Medium	High	Reduce threshold to 0.15; widen layer range to last 8; verify find_document_token_span() indices
False positives on function words	Medium	Medium	max_consecutive_failures=3 absorbs isolated low-attention tokens
Storage exhaustion	Medium	Medium	Weekly purge of old checkpoints; use persist_directory to avoid re-embedding
CodecSep GPU conflict	High	Low	Schedule ASQA evaluations overnight; daytime reserved for small-batch debug only
transformers API changes	Low	High	Pin to 4.46.0; check out.attentions is not None before proceeding
ASQA dataset schema changes	Low	Medium	Pin datasets==3.0.2; validate field names after download

---

Week-by-Week Schedule

Week	Activity	Milestone / Go/No-Go Check
1	Environment setup; CUDA verification; `00_smoke_test.py` passes	VRAM under 8 GB with model loaded; `output_attentions` returns non-None
2	ChromaDB vector store; hybrid retriever; Vanilla RAG end-to-end	Retrieval returns ≥ 1 relevant doc on 10 ASQA samples
3	Implement `attention.py`; all `test_attention.py` tests pass on CPU	Grounding scores computed correctly; `find_document_token_span` verified
4	Implement AttributedRAGDecoder; test_decoder.py passes	Abort fires correctly on synthetic low-grounding prompts; VRAM within budget
5	Threshold calibration (`01_calibrate_threshold.py`) on `calib_set`; ablation sweep	Optimal threshold selected; `fig1_ablation_curve.pdf` plotted
6	Overnight: ASQA full eval + finance corpus eval	Primary metrics computed for both datasets; JSON results saved
7	Statistical tests; bootstrap CIs; `fig2_grounding_distribution.pdf`; paper draft	Paper draft complete; all tables and figures finalised

---

Deliverables Checklist

- [ ] `src/rag_attribution/` Python library — fully docstrung, passes `pytest tests/ -v`
- [ ] `data/finance_corpus/documents.parquet` + `queries.parquet` — serialised, reproducible
- [ ] `outputs/results/asqa_constrained.json` — full per-query results
- [ ] `outputs/results/asqa_baseline.json` — full per-query results
- [ ] `outputs/results/ablation_sweep.csv` — threshold sweep data
- [ ] `outputs/figures/fig1_ablation_curve.pdf` — precision-recall tradeoff curve
- [ ] `outputs/figures/fig2_grounding_distribution.pdf` — grounding score histogram
- [ ] Final research paper PDF — methodology, OOM analysis, ablation, Wilcoxon test, 3+ qualitative abort examples

---

Explicit Scope Constraints

Hard boundary: zero fine-tuning. If the model struggles to clear the 0.3 threshold, the correct response is to adjust the thresholding algorithm, the layer range, or the input format — not to initiate a QLoRA run. Fine-tuning would derail the timeline and constitutes scope creep.

Out of scope this semester:

- Multi-hop reasoning (queries requiring chaining across > 1 document)
- Retrieval optimisation (ChromaDB retriever is fixed after Week 2)
- Generalisation to model families other than Llama-3-8B-Instruct
- Deployment / serving infrastructure (FastAPI, Docker, etc.)

References

1. [PDF] Metamorphic Testing for Hallucination Detection in RAG Systems
2. HalluRAG-RUG at SemEval-2025 Task 3: Using Retrieval- ...